

Лабораторна робота №4

Система прийняття рішень на основі нечітких правил для моделювання експертних систем

Роботу виконав:

Грунда Ярослав

Студент 3-го курсу

Групи ФІ-21

Загальний опис роботи

У даній роботі реалізовано нечітку систему прийняття рішень, яка визначає необхідну кількість кави програмісту залежно від його стану. Система враховує різні параметри, що відображають фізичний, психологічний та робочий стан людини, і на основі нечітких правил визначає рекомендовану дозу кофе.

Вхідні параметри:

1. **no_sleep_hours** (0–36 год) — кількість годин без сну.
 - a. fresh: trapmf(0, 0, 6, 10)
 - b. tired: gbellmf(4, 5, 14)
 - c. zombie: gaussmf(24, 3)
2. **coding_hours** (0–36 год) — скільки годин людина вже кодить.
 - a. short: zmf(0, 6)
 - b. medium: trimf(6, 8, 12)
 - c. marathon: smf(12, 16)
3. **task_difficulty** (1–10) — складність задачі.
 - a. easy: trapmf(1, 1, 3, 5)
 - b. medium: gbellmf(1.3, 3, 5)
 - c. brainfuck: gaussmf(9.5, 1)
4. **brain_crash** (1–10) — ментальне вигорання.
 - a. stable: zmf(1, 3)
 - b. fragile: gbellmf(1.5, 3, 6)
 - c. panic: smf(7, 10)
5. **deadline_pressure** (0–10) — наскільки горить дедлайн.
 - a. none: trimf(0, 0, 2)
 - b. soon: gbellmf(2, 4, 5)
 - c. urgent: gaussmf(9, 1)

Вихідна змінна:

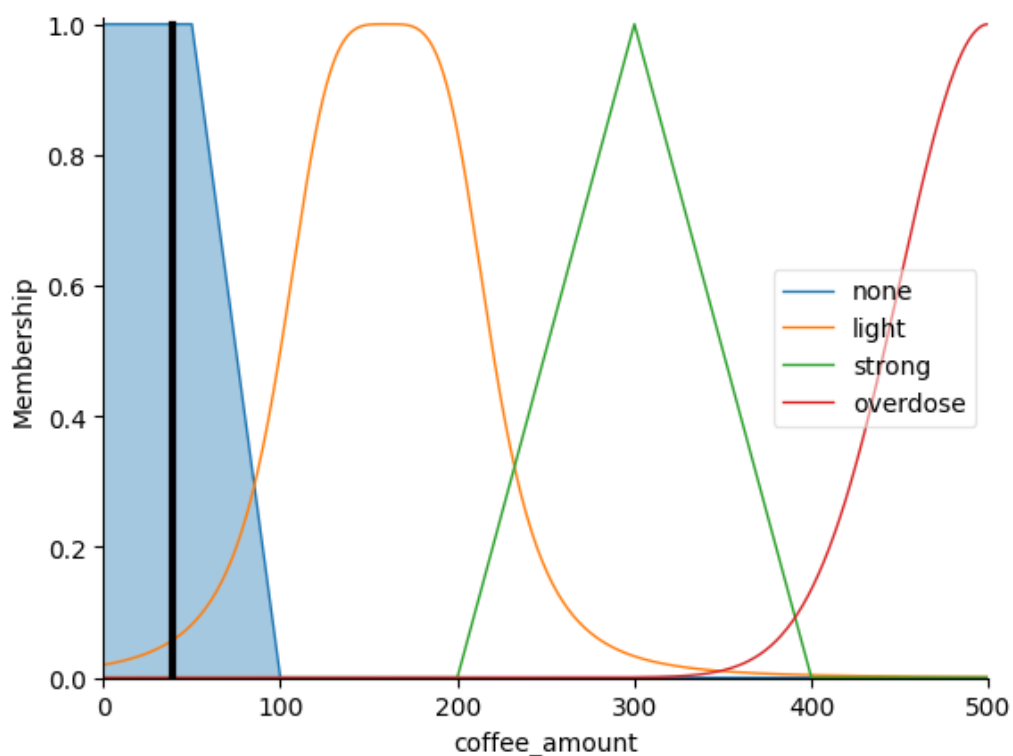
- **coffee_amount** (0–500 мл) — рекомендована кількість кави.
 - none: trapmf(0, 0, 50, 100)
 - light: gbellmf(60, 2, 160)
 - strong: trimf(200, 300, 400)
 - overdose: gaussmf(500, 50)

Опис правил:

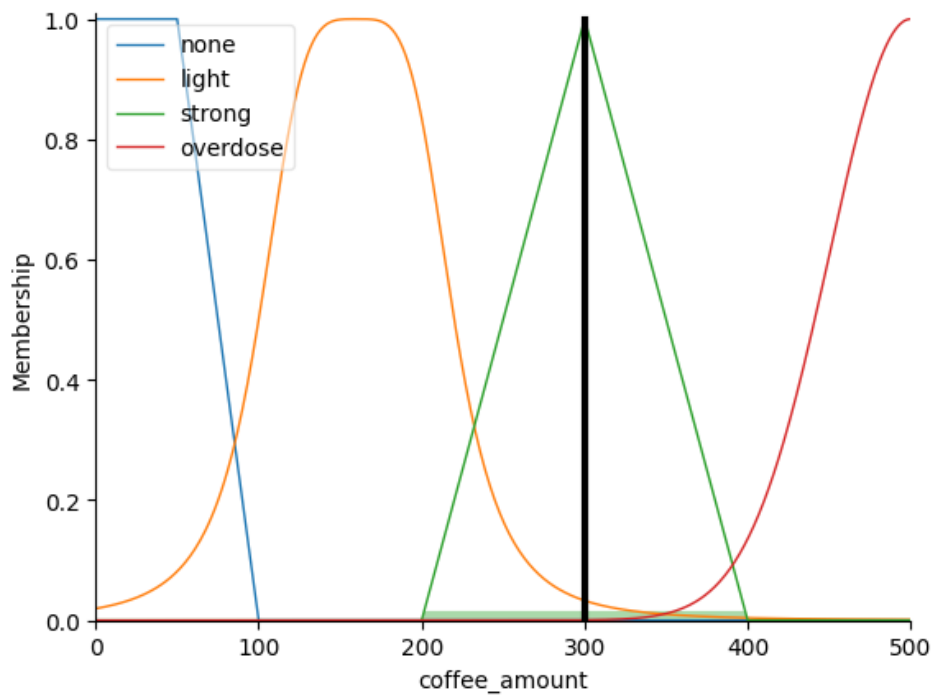
1. Якщо не спав недавно, завдання легке, і стан стабільний — кави не треба.
2. Якщо втомлений, завдання середнє, і працює кілька годин — легка кава.
3. Якщо зомбі або у стані паніки — overdose.
4. Якщо дедлайн горить і стан нестабільний — strong.
5. Якщо brainfuck і marathon — overdose.
6. Якщо tired + soon + fragile — strong.
7. Якщо тільки сів, задача легка, ще свіжий — none.
8. Якщо дедлайну нема, стабільний, нещодавно спав — none.
9. Якщо marathon, середня задача, але стан стабільний — light.
10. Якщо зомбі, дедлайну нема, але стабільний — strong.

Результати (приклади наборів):

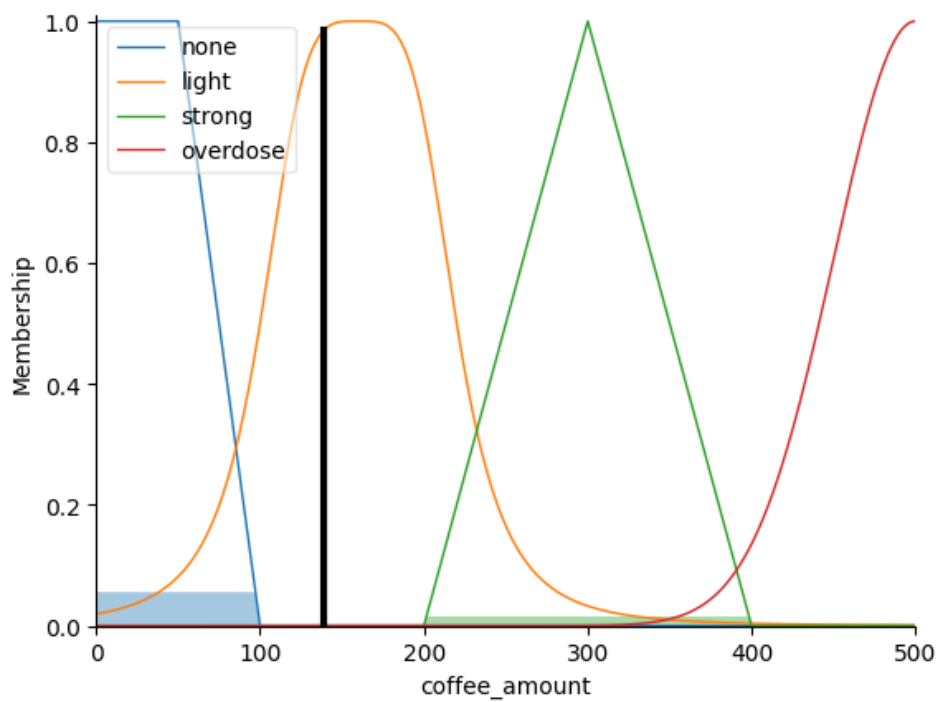
1. **sleep=1, coding=1, task=1, crash=1, deadline=1** → coffee = 39 мл



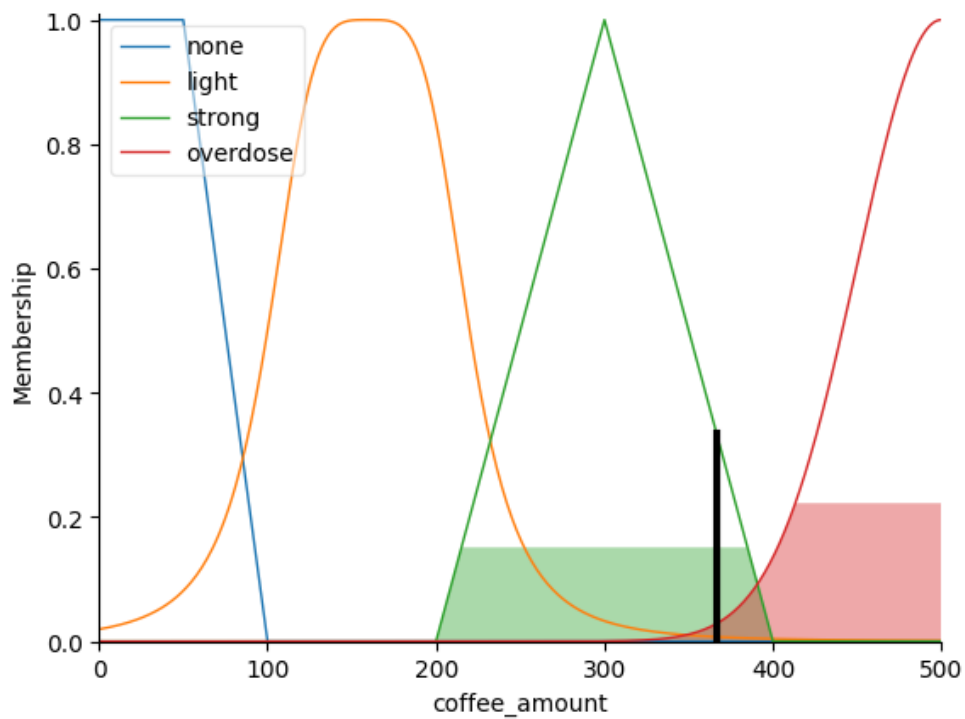
2. **sleep=6, coding=4, task=6, crash=3, deadline=7** → coffee = 300 мл



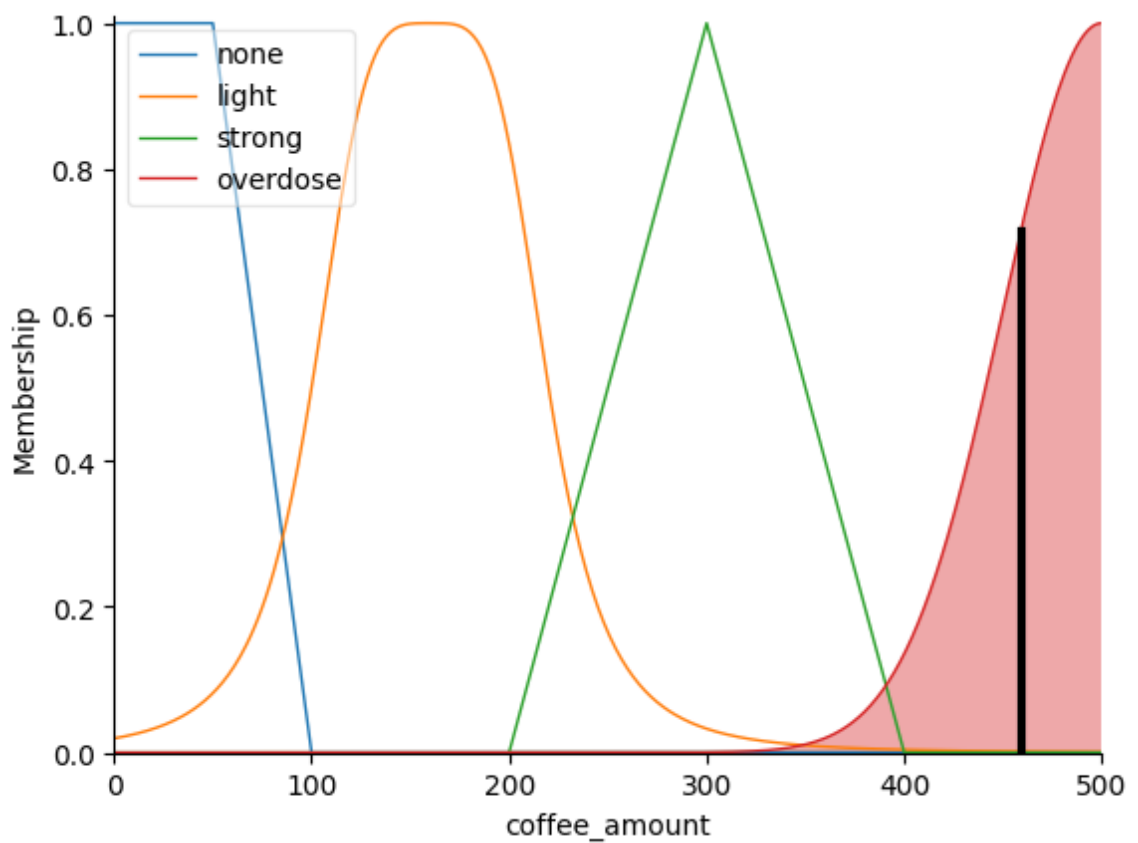
3. **sleep=9, coding=5, task=4, crash=3, deadline=6** → coffee = 139 мл



4. **sleep=18, coding=12, task=6, crash=8, deadline=8** → coffee = 366 мл



5. **sleep=36, coding=36, task=10, crash=10, deadline=10** → coffee = 459 мл



Висновки:

Нечітка логіка дозволяє будувати системи прийняття рішень у ситуаціях, де відсутні чіткі межі між станами. Завдяки цьому система може імітувати «людське» мислення та адаптувати вихід до багатьох факторів одночасно. В нашій задачі це дозволяє грамотно дозувати каву на основі психофізичного стану програміста.

Основною перевагою нечітких систем є здатність імітувати гнучке, "людське" мислення в умовах нечіткої, неповної інформації.

Водночас, одним з головних викликів при побудові таких систем є процес формулювання правил.

Додаткові приклади застосування нечіткої логіки:

- Розумне освітлення у будинку: яскравість світла залежить від часу доби, природного освітлення та активності
- Автоматичне керування кондиціонером: температура + вологість + активність → режим роботи
- Система медичного моніторингу: пульс + тиск + втома → режим лікування або рекомендація
- Геймдизайн: адаптація складності гри залежно від часу гри, помилок і прогресу гравця

Наприклад (до системи Genesis, яка повертає ворогів з часом (гра Middle-earth: Shadow of War):

Вхідні змінні (характер ворога / ставлення до гравця):

- ☐ respect → наскільки він тебе поважає
- ☐ fear → страх перед гравцем
- ☐ revenge_desire → бажання помсти
- ☐ injury_level → наскільки ти його калічив

Вихід:

- ☐ aggression_level → як ворог себе поводить (тікає, кидається в атаку, кличе підмогу)

